Проект. Модуль Exceptions [PDF]

Финальный проект модуля «Исключения»

Ниже описан итоговый проект по всему модулю **«Исключения»**, который позволит вам применить на практике все ключевые темы, изученные в уроках. Примерное время выполнения: **3–5 часов**, учитывая, что вы будете детально разбираться с кодом и обработкой ошибок.

Описание проекта

Вам предстоит создать **консольное приложение**, которое хранит **данные о книгах** (или других сущностях по вашему выбору, например, фильмы, товары и т. п.) **в оперативной памяти** и даёт возможность пользователю:

1. **Посмотреть** список всех доступных книг.
2. **Выдать** (арендовать) книгу, если она есть в наличии.
3. **Вернуть** (освободить) книгу.
4. **Добавить** новую книгу в систему.
5. **Выйти** из приложения.

При этом должны применяться **стандартные исключения** Java, а также **кастомные (собственные)** для различных ситуаций, связанных с бизнес-логикой (например, «Нет свободных экземпляров»).

Вы будете **перехватывать** и **обрабатывать** эти исключения, чтобы программа не «падала», а корректно информировала пользователя об ошибках и продолжала работу.

Необходимые навыки для реализации

1. **Основы синтаксиса Java**: классы, методы, условные операторы, циклы.
2. **Понимание иерархии исключений** (checked и unchecked), умение применять:
   - `try-catch`
   - `throw` и `throws`
   - `finally`
   - (опционально) `try-with-resources` (например, при работе со `Scanner`).
3. **Умение создавать собственные классы исключений** и продумывать их логику.
4. **Работа с консолью** (ввод пользователя) и базовыми коллекциями/массивами, чтобы хранить данные в памяти.
   - Можно использовать `ArrayList`, обычные массивы `[]` или любую другую структуру, которая удобна вам.

(Все эти темы вы уже изучали в прошлых модулях и в текущем модуле.)

Детальное описание задания

Структура данных

Создайте класс `Book`. Минимальные поля:

- Название (`String title`).
- Автор (`String author`) или аналог (если это фильм — «режиссёр», если товар — «производитель»).
- Количество доступных экземпляров (`int availableCopies`).

Храните объекты в **списке (List)** или **массиве**, который будем называть «каталог» (например, `List<Book> catalog`).

Бизнес-логика (основные операции)

- **Показать список** всех объектов (вывод их на консоль).
- **Добавить новый объект**: пользователь вводит название, автора, количество; вы добавляете их в `catalog`.
- **Выдать объект** (например, «взять книгу»): пользователь вводит название.
  - Если нет такого объекта в каталоге, выбрасывайте и обрабатывайте либо **стандартное** (например, `NoSuchElementException`) или **кастомное** (`ItemNotFoundException`).
  - Если `availableCopies == 0`, выбросите **кастомное** исключение, например, `NoAvailableCopiesException`.
  - Если всё в порядке, уменьшаете `availableCopies` на 1.
- **Вернуть объект**: пользователь вводит название, вы находите объект в `catalog` и увеличиваете `availableCopies` на 1. Если объекта нет, тоже логируете это как ошибку.

Исключения

- **Обязательные**:
  1. **Кастомное исключение** (минимум одно, например, `NoAvailableCopiesException`).
  2. **Несколько блоков catch** для разных типов ошибок (например, неправильный ввод пользователя — `NumberFormatException` или `InputMismatchException`, «объект не найден» — ваш кастомный или стандартный).
- **Опционально**: при работе со `Scanner` (если хотите, можно использовать `try-with-resources`), чтобы автоматически закрыть ресурсы.

Консольный интерфейс

- Предусмотрите меню (цикл `while` с пунктами 1–5):
  1. Вывести каталог.
  2. Добавить объект.
  3. Выдать объект.
  4. Вернуть объект.
  5. Выйти из приложения.
- Очень желательно - не делать всю логику большой простыней кода, а разбивать весь код на методы
- Обрабатывайте некорректный ввод: если пользователь вводит нечисловое значение в пункте меню, вы улавливаете исключение (`InputMismatchException` или `NumberFormatException`), выводите сообщение и даёте возможность повторить ввод.

Без работы с файлами

- Согласно условию, не надо сохранять и загружать данные в/из файла. Всё хранится **в оперативной памяти**.
- При желании вы можете расширить эту функциональность, но это не обязательно для проекта.

Рекомендации по реализации (шаги)

Создайте классы:

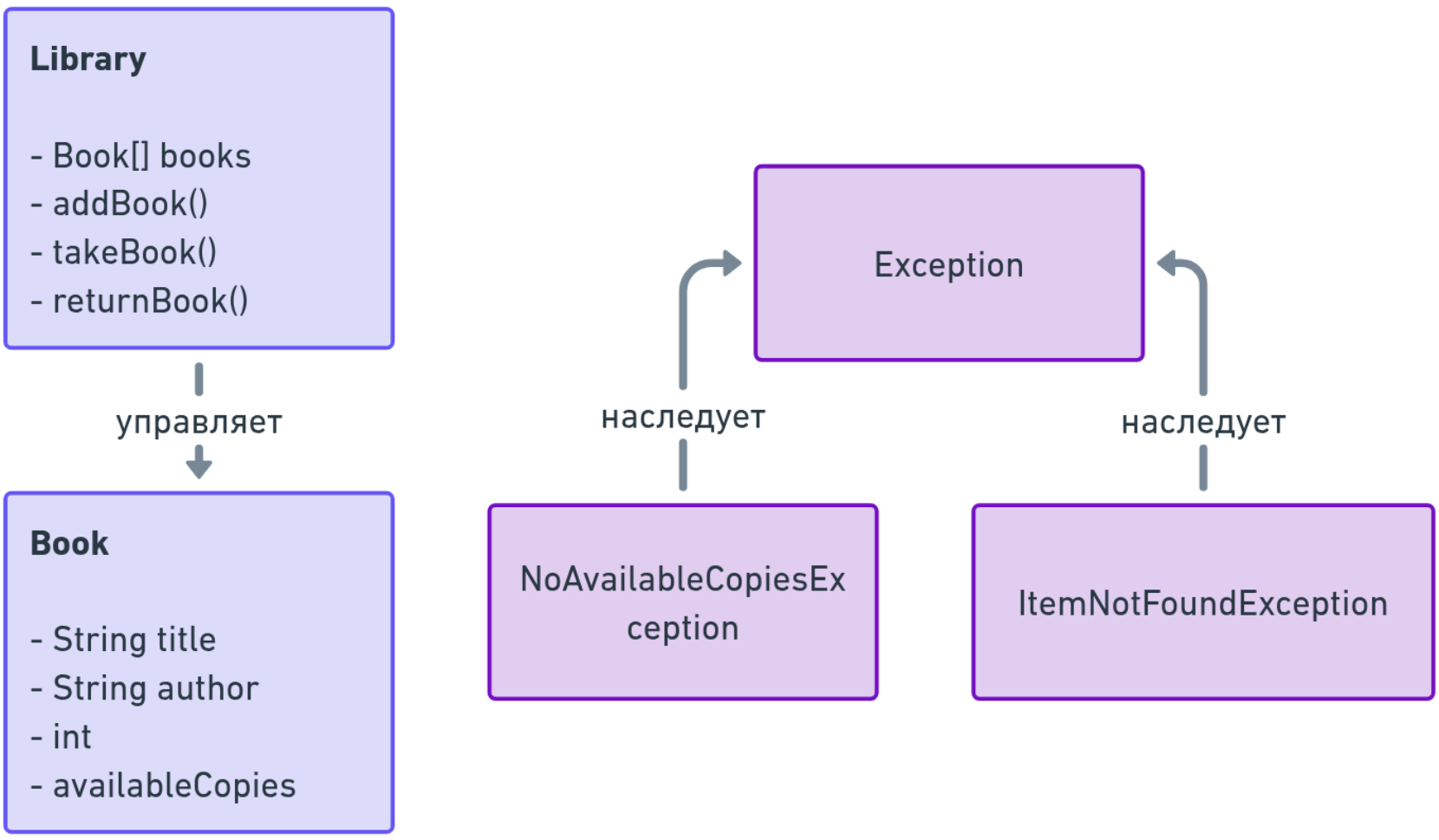
- `Book` (с полями `title`, `author`, `availableCopies`).
- Один-два **кастомных** класса исключений, например:

- `NoAvailableCopiesException` (unchecked или checked — на ваше усмотрение).
- `ItemNotFoundException` (если хотите разделить логику).

Сделайте класс `Library` или `CatalogManager`, где будет `List<Book> catalog` и методы:

- `public void addBook(String title, String author, int copies)`
- `public void takeBook(String title)` (уменьшает copies или бросает исключение)
- `public void returnBook(String title)` (увеличивает copies или бросает исключение)
- `public List<Book> getAllBooks()` (возвращает список всех книг)

Пример вашей структуры классов:



В `main` **методе**:

- Создайте объект `Library` (или как вы назовёте ваш «хранилище»).
- Создайте цикл `while(true)`, который выводит меню (1–5).
- Используйте `Scanner` для считывания выбора пользователя и данных для книг.
- Оборачивайте потенциально «опасные» операции в `try-catch`, чтобы перехватывать исключения (`NoAvailableCopiesException`, `ItemNotFoundException`, `InputMismatchException` и т. д.).

Не забудьте:

- При выброшенном исключении вывести сообщение пользователю на консоль, но не «ронять» программу.
- Если пользователь выбрал пункт «5», сделайте `break;` из цикла или завершите программу.

(Опционально): можно добавить расширенный функционал (например, несколько экземпляров одного автора, или поиск по ID), но помните, что проект должен оставаться в разумных рамках для **3–5 часов** практики.

Как проверить свою работу

1. **Различные сценарии**:
   - **Успешный сценарий**: пользователь добавляет несколько книг, затем успешно выдаёт/возвращает их.
   - **Ошибка: нет книги**: пользователь пытается выдать книгу, которой не существует → выбрасывается `ItemNotFoundException` (или стандартное `NoSuchElementException`).
   - **Ошибка: 0 экземпляров**: пользователь пытается выдать книгу, но `availableCopies = 0` → выброс `NoAvailableCopiesException`.
   - **Неверный выбор в меню**: пользователь вводит «abc» вместо пункта меню → `InputMismatchException` или `NumberFormatException`, вы выводите «Повторите ввод».
2. **Что должно произойти**:
   - При корректном вводе метод спокойно выполняется.
   - При ошибке вы перехватываете исключение, выводите сообщение, **не давая** программе «упасть». Пользователь может продолжить взаимодействие.
3. **Вывод на консоль**:
   - Убедитесь, что программа «рассказывает», что именно случилось (какое действие выполняется или почему не получилось).

Итог: Если ваше приложение корректно обрабатывает все эти ситуации и продолжает работать без аварийных падений, значит вы успешно реализовали все ключевые моменты, связанные с исключениями: **иерархию**, **обработку**, **кастомные исключения** и **ошибки ввода**.